Tecnologías de la Información

Universidad de Huelva

Tema 5

TAD Genéricos

INDICE

1. Introducción.
2. Plantillas de funciones o funciones genéricas.
3. Sintaxis de plantillas genéricas.
4. Especialización de funciones genéricas.
5. Implementación del TAD Lista Genérica.
6. Iteradores sobre listas.

TAD Genéricos

5.1 Introducción a la Generalidad

La generalidad es la capacidad de los compiladores de definir y generar tipos abstractos de datos sin indicar explícitamente el tipo de los datos que almacenan.

Veamos un ejemplo para comprender su necesidad de utilización: Supongamos que necesitamos construir una función **mayor** que nos devuelva el elemento más grande de entre dos elementos dados de cualquier tipo. Una posible solución es esta:

```
int mayor(int a, int b )
{
    if (a<b)
        return b;
    return a;
}
```

```
float mayor(float a, float b )
{
    if (a<b)
        return b;
    return a;
}
```

```
long mayor(long a, long b )
{
    if (a<b)
        return b;
    return a;
}
```

Como podemos observar aunque, el código es exactamente el mismo para las tres funciones, lo único que cambia es el tipo de los parámetros.

También hemos de observar que aunque hemos definido tres funciones, aún hay tipos de datos básicos para los cuales no se ha definido una función mayor.

Este tipo de problemas de programación es muy frecuente y normalmente se resuelve utilizando la sobrecarga de métodos y funciones como hemos visto en el ejemplo.

El problema principal de la sobrecarga es que haya que implementar tantas funciones con el mismo nombre y distintos tipos de datos. Además aunque definamos un conjunto de funciones o métodos sobrecargados, nadie nos asegura que se utilicen todos o que las que necesitemos estén implementados, por ejemplo, que se necesite la función mayor para dos polinomios, clientes o ventas, etc.

Este es el motivo principal por el que la utilización de la sobrecarga no nos sirve completamente y necesitamos que el lenguaje nos proporcione la sintaxis para definir funciones o métodos que no necesiten que al principio se indique el tipo de datos con el que tiene que tiene que trabajar.

Un ejemplo de función genérica podría ser el siguiente:

```
<TIPO> mayor(<TIPO> a, <TIPO> b )
{
  if (a<b)
    return b;
  return a;
}
```

Donde <TIPO> podría ser cualquier tipo de datos básico de C++ o estructurado que tuviera sobrecargado el operador <.

Al utilizar la función genérica con un determinado tipo para los parámetros, se genera automáticamente el código de la función para el tipo de parámetros especificado.

5.2 Funciones y Tipos Genéricos: Plantillas

- C++ permite definir funciones genéricas mediante el uso de plantillas (**Templates**). Ejemplo:

```
template <class TIPO>
TIPO mayor(TIPO a,TIPO b)
{
  if (a<b)
    return b;
  return a;
}
```

```
void main (void)
{
  int a=5, b=8, c;
  float d=3.0, e=5.1, f;
  c=mayor(a,b);
  f=mayor(d,e);
  cout << c << " "<<f;
}
```

- Una función genérica define un conjunto de operaciones que se aplicarán a diferentes tipos de datos.
- Una plantilla de funciones y tipos es como un patrón de código que se copia en el momento que se utiliza.
- Una plantilla de funciones especifica un conjunto infinito de funciones que pueden ser aplicadas a distintos tipos de datos.
- Una plantilla de tipos especifica un conjunto infinito de tipos.
- Una plantilla de funciones describe las propiedades genéricas de una función.
- Una plantilla de tipos describe los atributos genéricos (campos, atributos y métodos) de un tipo de datos estructurado.
- La única desventaja de la utilización de tipos genéricos es que el tamaño del código ejecutable crece cada vez que se utiliza una plantilla independiente de si ya se ha utilizado previamente.

5.2.1 **Sintaxis**

La definición de una plantilla para una función se realiza utilizando la cabecera **template <class TIPO_1, class TIPO_N>** antes del cuerpo de la función, donde:

template: Esta palabra indica que se va a definir una plantilla

class: Esta palabra indica que se van a utilizar las distintas clases de tipos genéricos declarados entre los símbolos < y > (también puede utilizarse la palabra reservada **typename**, con idéntico uso).

TIPO_1, **TIPO_N**: Tipos a utilizar dentro del cuerpo de la función.

Un ejemplo lo hemos visto en la implementación de la función mayor (página anterior).

La definición de una plantilla para un tipo estructurado como puede ser una estructura, se antepone igualmente la cabecera **template <class TIPO_1, class TIPO_N>** antes de la definición de la estructura. Un ejemplo:

```
template <class T>
struct TNodo_Datos
{
  T Datos;   //Datos a almacenar en cada nodo
  TNodo_Datos *Siguiente; //Puntero al siguiente nodo
};
```

La forma de declarar variables estructuras de tipo TNodo_Datos que almacenen distintos tipos de datos se realiza acompañando al nombre del tipo estructura, el nombre del tipo de datos que almacena encerrados entre los símbolos < y >. Un ejemplo:

```
void main (void)
{
  TNodo_Datos<int> a; // a es una estructura que contiene un entero
                      //como dato;
  TNodo_Datos<float> *b;     //b es un puntero a un TNodo_Datos que
                             //contiene un real

  a.Datos=50;    //Datos es un campo de tipo entero
  a.Siguiente=NULL;

  b=new TNodo_Datos<float>;
  if (b!=NULL)
  {
    b->Datos=56.5;           //Datos es un campo de tipo real
    b->Siguiente=NULL;
  }
  delete b;
  b=NULL;
}
```

La única diferencia entre un tipo genérico y un tipo no genérico, es que al primero se le indica además del nombre del tipo, el tipo o tipos de datos que almacenan entre < >.

La definición de una plantilla para una clase se realizar también anteponiendo la cabecera **template <class TIPO_1, class TIPO_N>** antes del cuerpo de la clase y de cada método de la misma que se implemente fuera de la definición de la clase. Un ejemplo:

```
template <class T>
class TLista
{
       TNodo_Datos<T> *elementos;//Puntero al primer nodo de la lista.
       int  n;                   //nº de nodos que tiene la lista
       void DestruirLista(void);
       TNodo_Datos<T> *Anterior(int index);
    public:
       TLista();
       ~TLista();
       void insertar(int index, T elto);
```

```
        int longitud();
        int posicion(T elto);
        T observar(int index);
        bool esvacia();
        void modificar(int index, T elto);
        void eliminar(int index);
};
```

Para implementar cada método de la clase, además de especificar el mismo **template** que para la definición de la clase, hay que indicar en el nombre de la clase que antecede al nombre del método, el tipo de datos que utiliza en la declaración de la clase entre los símbolos < y >. Un ejemplo:

****Nota Importante****: La declaración de los métodos se realiza en el mismo fichero .h que la clase y **NO** en un fichero .cpp. Esto es debido a que se está definiendo mediante una plantilla un tipo genérico. Si intentamos poner el código de los métodos genéricos en un fichero .cpp algunos compiladores dan el error que no encuentran la declaración de los métodos.

```
template <class T>
TLista<T>::TLista() {
     // Código del constructor de la clase Lista Genérica
}

template <class T>
TLista<T>::~TLista() {
     // Código del destructor de la clase Lista Genérica
}
template <class T>
TNodo_Datos<T> * TLista<T>::Anterior(int index) {
     // Código del método anterior de la clase Lista Genérica.
};

template <class T>
void TLista<T>::insertar(int index, T elto) {
     // Código del método insertar de la clase Lista Genérica.
}
```

5.2.2 Ejemplos

En el siguiente ejemplo podemos ver una función genérica que calcula el máximo valor de un vector de datos de cualquier tipo.

```
template <class TIPO>
TIPO mayor(TIPO v[ ], int dim)
{
  TIPO aux;
  aux = v[0];
  for (int i =1; i<dim; i++)
  {
    if ( v[i]>aux )
      aux = v[i];
  }
  return aux;
}
void main()
{
  int w[5]={1,7,3,4,5};
  float k[4]={4.6,7.8,3.2,4.0};
  char c[]={'f','e','t','h'};
  int a;
  float b;
  char m;
  a = mayor(w, 5);
  b = mayor(k, 4);
  m = mayor(c, 4);
}
```

5.3 Sobrecarga de Plantillas de Funciones

Las plantillas de funciones se pueden definir para distintas combinaciones de parámetros, es decir, se pueden sobrecargar como las funciones y los métodos.

Por ejemplo, para la función **mayor(),** serían válidas las siguientes plantillas sobrecargadas

```
template <class T>
T mayor ( T a, T b ) {
   // Obtiene el mayor de los parámetros a y b
};
template <class T>
T mayor ( T a, T b, T c){
   // Obtiene el mayor de los parámetros a, b y c
};

template <class T>
T mayor (T v[ ], int dim){
   //Código que obtiene el mayor de los elementos del vector v
};
```

5.4 Especialización de Funciones Genéricas

La existencia de una plantilla no impide que se pueda definir una función normal que prevalezca sobre la definición de la plantilla.

Por ejemplo, hemos visto en los ejemplos de los apartados anteriores cómo declarar la plantilla para calcular el mayor de dos valores. Esta plantilla funciona bien para cualquier tipo de datos básico y estructurado siempre y cuando no sea una tabla de caracteres. En este caso podemos especializar esta función mayor para cadenas y generalizarla para el resto de tipos. Ejemplo:

```
template <class TIPO>
TIPO mayor(TIPO a, TIPO b )
{
  if (a<b)
    return b;
  return a;
}

char* mayor (char* s1, char* s2)
{
if (strcmp(s1,s2)>0)
  return s1;
return s2;
}
```

Las reglas que utiliza C++ para encontrar la función correcta es:

- Buscar una coincidencia exacta de la función entre todas las funciones declaradas.
- Buscar una plantilla de función que más se le adapte.
- Si no encuentra coincidencia, se genera un error.

5.5 TAD Lista Genérica

```
template <class T>
struct TNodo_Datos {
  T Datos;   //Datos a almacenar en cada nodo
  TNodo_Datos *Siguiente; //Puntero al siguiente nodo
};
```

```
template <class T>
class TLista
{
     TNodo_Datos<T> *elementos; //Puntero al primer nodo.
     int  n;                    //nº de nodos que tiene la lista

     TNodo_Datos<T> *Anterior(int index);
   public:
     TLista();
     ~TLista();
     void insertar(int i, T elto);
     int longitud();
     int posicion(T elto);
     T observar(int i);
     bool esvacia();
     void modificar(int i, T elto);
     void eliminar(int i);
};
template <class T>
TLista<T>::TLista()
{
   elementos=NULL;
   n=0;
}

template <class T>
TLista<T>::~TLista()
{
   TNodo_Datos<T> *Nodo_Borr=elementos, *Nodo_Sig;

   while (Nodo_Borr!=NULL) {
      Nodo_Sig=Nodo_Borr->Siguiente;
      delete Nodo_Borr;
      Nodo_Borr=Nodo_Sig;
   }
   elementos=NULL;
   n=0;
}

template <class T>
TNodo_Datos<T> * TLista<T>::Anterior(int i)
{
   TNodo_Datos<T> *Nodo_Aux=elementos,*Nodo_Ant=NULL;
   int v=1;
   if (Nodo_Aux!=NULL)
      while (Nodo_Aux!=NULL && v<i)
      {
           Nodo_Ant=Nodo_Aux;
           Nodo_Aux=Nodo_Aux->Siguiente;
           v++;
      }
   return Nodo_Ant;
};
```

```
template <class T>
void TLista<T>::insertar(int i, T elto)
{
   TNodo_Datos<T> *Nodo_Aux=new TNodo_Datos<T>, *Nodo_Ant;
   if (Nodo_Aux!=NULL)
   {
      Nodo_Aux->Datos=elto;
      Nodo_Aux->Siguiente=NULL;
      Nodo_Ant = Anterior(i);
      if (Nodo_Ant==NULL){
         Nodo_Aux->Siguiente=elementos;
         elementos=Nodo_Aux;
      }
      else {
         Nodo_Aux->Siguiente=Nodo_Ant->Siguiente;
         Nodo_Ant->Siguiente=Nodo_Aux;
      }
      n++;
   }
}

template <class T>
void TLista<T>::eliminar(int i)
{
   TNodo_Datos<T> *Nodo_Ant, *Nodo_Aux;
   Nodo_Ant=Anterior(i);

   if (Nodo_Ant==NULL)
   {
      Nodo_Aux=elementos;
      elementos=Nodo_Aux->Siguiente;
   }
   else
   {
      Nodo_Aux=Nodo_Ant->Siguiente;
      Nodo_Ant->Siguiente=Nodo_Aux->Siguiente;
   }
   delete Nodo_Aux;
   n--;
};

template <class T>
void TLista<T>::modificar(int index, T elto)
{
   TNodo_Datos<T> *Nodo_Ant=Anterior(i), *Nodo_Aux;
   if (Nodo_Ant==NULL)
      Nodo_Aux=elementos;
   else
      Nodo_Aux=Nodo_Ant->Siguiente;
   Nodo_Aux->Datos=elto;
}
```

```
template <class T>
T TLista<T>::observar(int i)
{
   TNodo_Datos<T> *Nodo_Ant=Anterior(index), *Nodo_Aux;
   if (Nodo_Ant==NULL)
      Nodo_Aux=elementos;
   else
      Nodo_Aux=Nodo_Ant->Siguiente;
   return Nodo_Aux->Datos;
}

template <class T>
int TLista<T>::posicion(T elto)
{
   TNodo_Datos<T> *Nodo_Aux=elementos;
   bool encontrado=false;
   int v=1;

   while (Nodo_Aux!=NULL && !encontrado)
      if (Nodo_Aux->Datos!=elto)
      {
         Nodo_Aux=Nodo_Aux->Siguiente;
         v++;
      }
      else
        encontrado=true;
   return (encontrado ? v : -1);
}

template <class T>
bool TLista<T>::esvacia()
{
  return (n == 0);
}

template <class T>
int TLista<T>::longitud()
{
   return n;
}
```